Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут» імені Ігоря Сікорського
Факультет інформатики та обчислювальної техніки
Кафедра автоматики та управління в технічних системах

Лабораторна робота № 1

із дисципліни: «Сучасні технології розробки WEB-застосувань з використанням мови Python»

на тему: «Використання фреймворку FastAPI та SQLAlchemy ORM для створення вебзастосунку мовою Python»

Виконав:
студент групи ІА-34
Чунаков А.Р.
Перевірив:
Белоус Роман Володимирович

Київ-2026

Завдання. Для заданої предметної галузі розробити RESTfull API вебзастосунок

1) Контент повинен генеруватися на сервері і у вигляді HTML відправляється на клієнт.

2) Дані мусять зберігатися у реляційній БД, наприклад, SQLite. Схема даних повинна містити щонайменша 3 сутності.

3) Для доступу до даних слід використовувати SQLAlchemy ORM.

4) Передбачити режими роботи акторів двох ролей: адміністратор та користувач.

5) Хоча б для однієї сутності реалізувати усі чотири CRUD-операції (create, read, update, delete).

6) Внести зміни в OpenAPI документацію за замовченням.

Варіант 14

ІС «Замовлення їжі»

Інформаційна система призначена для автоматизації процесу взаємодії між клієнтом та рестораном шляхом надання вебінтерфейсу для перегляду актуального меню, формування замовлень у кошику та адміністративного управління контентом. Система забезпечує збереження даних про користувачів, страви та замовлення у реляційній базі даних, дозволяючи звичайним користувачам ознайомлюватися з асортиментом, а адміністраторам — виконувати повний цикл CRUD-операцій для оновлення переліку страв, зміни цін та контролю статусів замовлень. Завдяки використанню серверного рендерингу та ORM-технологій, система гарантує цілісність інформації, швидкий доступ до актуального меню.

1. Серверна генерація контенту (SSR)

Для забезпечення генерації контенту на стороні сервера було використано вбудований у FastAPI механізм інтеграції з шаблонізатором Jinja2. Замість передачі сирих даних у форматі JSON, сервер обробляє HTML-шаблони, вшиваючи в них дані з бази даних безпосередньо перед відправкою клієнту. Такий підхід дозволяє браузеру отримувати повністю сформовану сторінку, що покращує швидкість відображення первинного контенту та спрощує взаємодію з вебзастосунком без необхідності складного клієнтського рендерингу.

view-source:127.0.0.1:8000

Переносить строки

```html
<!DOCTYPE html>
<html lang="uk">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Меню ресторану | ІС «Замовлення їжі»</title>
    <style>
        :root {
            --primary: #2c3e50;
            --accent: #e67e22;
            --light: #f8f9fa;
        }
        body { font-family: 'Segoe UI', sans-serif; background-color: #f4f4f4; margin: 0; padding: 20px; }
        .container { max-width: 800px; margin: 0 auto; }
        header { display: flex; justify-content: space-between; align-items: center; margin-bottom: 30px; }

        .menu-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 20px; }
        .menu-item {
            background: white;
            padding: 20px;
            border-radius: 12px;
            box-shadow: 0 4px 6px rgba(0,0,0,0.05);
            transition: transform 0.2s;
        }
        .menu-item:hover { transform: translateY(-5px); }
        .price { color: var(--accent); font-weight: bold; font-size: 1.2em; }

        .btn-cart {
            background: var(--primary); color: white; border: none;
            padding: 10px 15px; border-radius: 6px; cursor: pointer; width: 100%;
            margin-top: 10px; font-weight: bold;
        }
        .btn-cart:hover { background: #34495e; }
        .admin-link { color: #7f8c8d; text-decoration: none; font-size: 0.9em; }
    </style>
</head>
<body>

<div class="container">
    <header>
        <h1>🍴 Наше Меню</h1>
        <a href="/admin" class="admin-link">Увійти як адмін</a>
    </header>

    <div class="menu-grid">

        <div class="menu-item">
            <h3>борщ</h3>
            <p class="price">150.0 грн</p>

            <form action="/add_to_cart" method="post">
                <input type="hidden" name="dish_id" value="1">
                <input type="hidden" name="user_id" value="1">
                <button type="submit" class="btn-cart">🛒 Додати у кошик</button>
            </form>
        </div>

        <div class="menu-item">
            <h3>чизкейк</h3>
            <p class="price">210.0 грн</p>

            <form action="/add_to_cart" method="post">
                <input type="hidden" name="dish_id" value="2">
                <input type="hidden" name="user_id" value="1">
                <button type="submit" class="btn-cart">🛒 Додати у кошик</button>
            </form>
        </div>

    </div>
```

Рис. 1.1 - Контент генерується на сервері і у вигляді HTML відправляється на клієнт.

2. Проєктування реляційної бази даних

Зберігання даних реалізовано за допомогою реляційної СУБД SQLite, яка інтегрована у проєкт у вигляді локального файлу food_app.db. Схема бази даних розроблена з дотриманням нормальних форм і включає три ключові сутності: «Користувачі» (Users) для керування доступом, «Страви» (Dishes) для формування меню та «Замовлення» (Orders), що виступає сполучною ланкою між клієнтом і контентом. Таку структуру обрано для забезпечення цілісності даних та можливості масштабування системи в майбутньому.

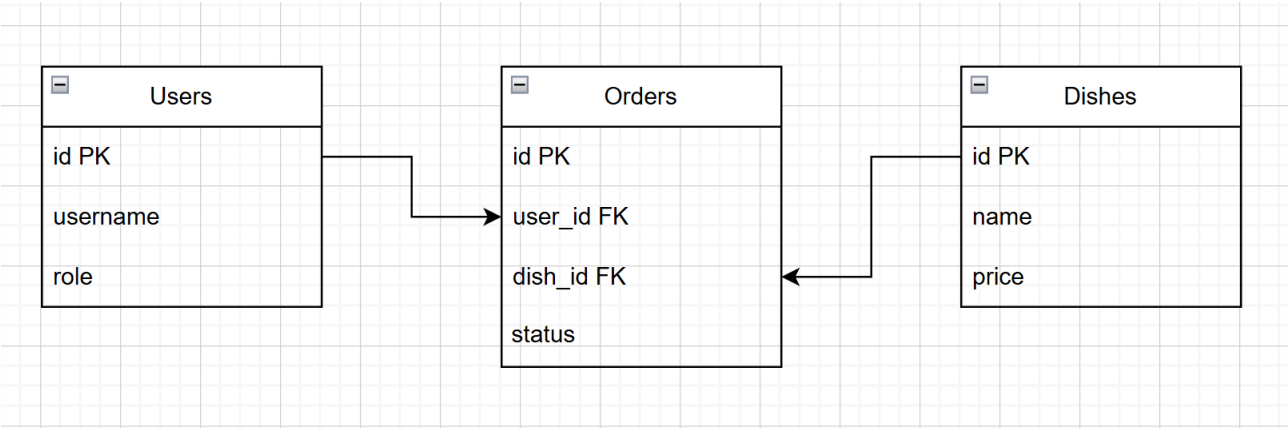

Рис 1.2 – ER-діаграма структури бази даних

Tables | Database Metadata

Tables		
dishes		
id	INTEGER	"id" INTEGER NOT NULL
name	VARCHAR	"name" VARCHAR
price	FLOAT	"price" FLOAT
description	VARCHAR	"description" VARCHAR
orders		
id	INTEGER	"id" INTEGER NOT NULL
user_id	INTEGER	"user_id" INTEGER
dish_id	INTEGER	"dish_id" INTEGER
status	VARCHAR	"status" VARCHAR
users		
id	INTEGER	"id" INTEGER NOT NULL
username	VARCHAR	"username" VARCHAR
role	VARCHAR	"role" VARCHAR

Рис 1.3 – Структура файлу бази даних

3. Використання SQLAlchemy ORM

Взаємодія з базою даних здійснюється виключно через об’єктно-реляційне відображення (ORM) за допомогою бібліотеки SQLAlchemy. Це дозволило абстрагуватися від написання прямих SQL-запитів і працювати з таблицями БД як зі звичайними об'єктами Python. У проєкті визначено декларативні моделі класів, що описують структуру таблиць, а виконання операцій читання та запису відбувається через сесії SQLAlchemy, що гарантує безпеку та чистоту коду.

```python
from sqlalchemy import create_engine, Column, Integer, String, ForeignKey, Float
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, relationship

DATABASE_URL = "sqlite:///./food_app.db"

engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    username = Column(String, unique=True)
    role = Column(String)  # 'admin' або 'user'

7 usages
class Dish(Base):
    __tablename__ = "dishes"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String)
    price = Column(Float)
    description = Column(String)

1 usage
class Order(Base):
    __tablename__ = "orders"
    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id"))
    dish_id = Column(Integer, ForeignKey("dishes.id"))
    status = Column(String, default="Pending")

Base.metadata.create_all(bind=engine)
```

Рис 1.4 – Структура файлу database.py з використанням SQLAlchemy

4. Рольова модель акторів У системі реалізовано розмежування прав доступу для двох типів акторів: адміністратора та користувача. Роль користувача передбачає перегляд меню та взаємодію з інтерфейсом замовлення. Роль адміністратора відкриває доступ до спеціалізованої панелі управління, де доступні інструменти для модифікації бази даних, що дозволяє керувати асортиментом страв та стежити за замовленнями, забезпечуючи адміністративний контроль над ресурсом.

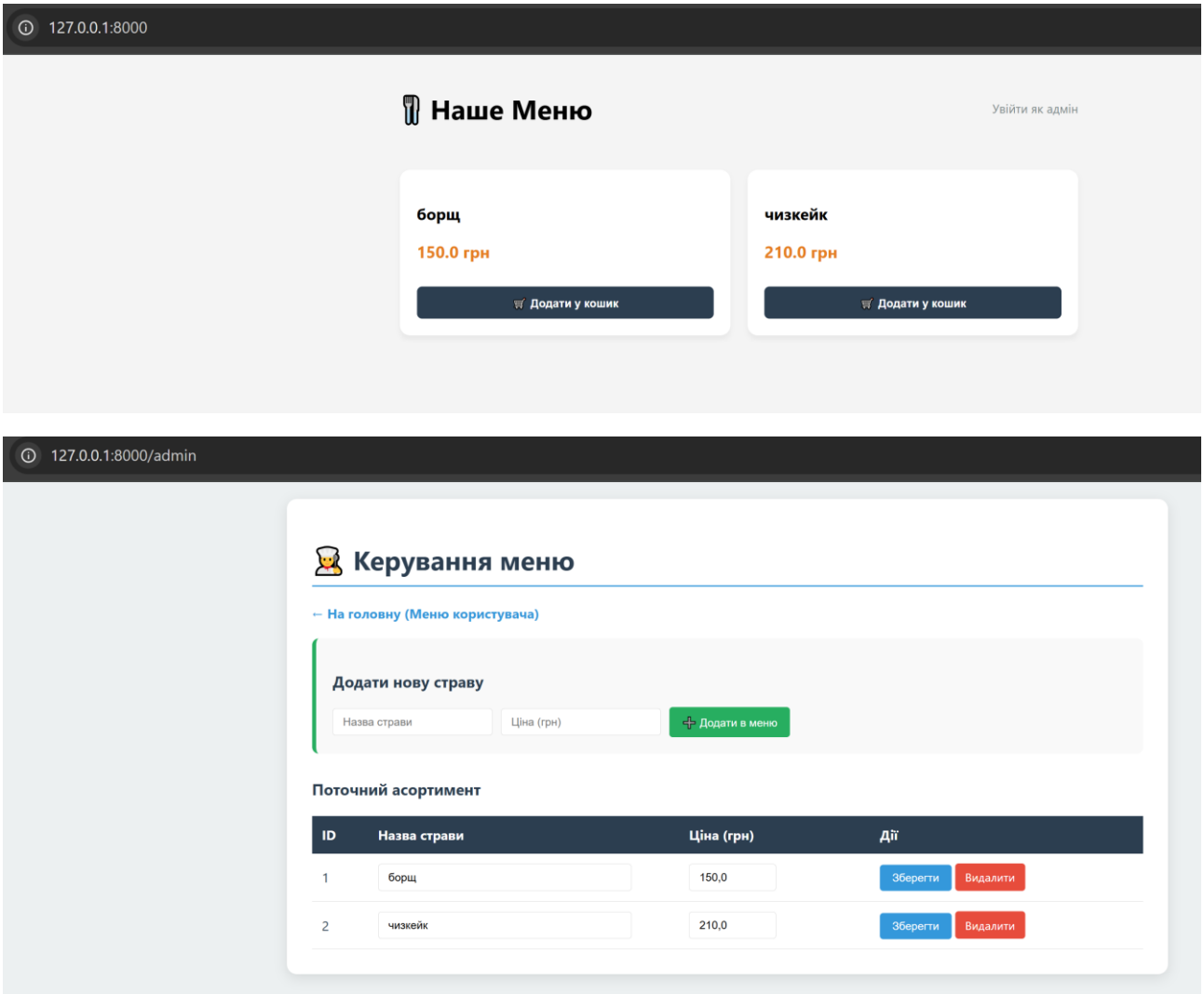

Рис 1.5 – Структура сторінок клієнта та адміна

5. Реалізація CRUD-операцій

Для сутності Dish було розроблено повний цикл CRUD-операцій, що дозволяє здійснювати комплексне управління даними. Адміністратор системи має можливість створювати нові записи Create, переглядати поточний список страв Read, редагувати назви та ціни існуючих позицій Update, а також видаляти неактуальні страви з меню Delete. Всі операції синхронізовані з базою даних через ORM та негайно відображаються в інтерфейсі користувача.

```python
# --- МАРШРУТИ АДМІНІСТРАТОРА (CRUD для Dish) ---

@app.get( path: "/admin", response_class=HTMLResponse)
def admin_panel(request: Request, dbs: Session = Depends(get_db)):
    dishes = dbs.query(db.Dish).all()
    return templates.TemplateResponse("admin.html", {"request": request, "dishes": dishes})

@app.post("/admin/dish/create")
def create_dish(name: str = Form(...), price: float = Form(...), dbs: Session = Depends(get_db)):
    new_dish = db.Dish(name=name, price=price)
    dbs.add(new_dish)
    dbs.commit()
    return RedirectResponse(url="/admin", status_code=303)

@app.post("/admin/dish/update/{dish_id}")
def update_dish(dish_id: int, name: str = Form(...), price: float = Form(...), dbs: Session = Depends(get_db))
    dish = dbs.query(db.Dish).filter(db.Dish.id == dish_id).first()
    dish.name = name
    dish.price = price
    dbs.commit()
    return RedirectResponse(url="/admin", status_code=303)

@app.get("/admin/dish/delete/{dish_id}")
def delete_dish(dish_id: int, dbs: Session = Depends(get_db)):
    dish = dbs.query(db.Dish).filter(db.Dish.id == dish_id).first()
    dbs.delete(dish)
    dbs.commit()
    return RedirectResponse(url="/admin", status_code=303)
```

Рис 1.6 – Структура файлу реалізації CRUD операцій

6. Кастомізація OpenAPI документації

Стандартну документацію Swagger UI, яка генерується FastAPI автоматично, було адаптовано під вимоги конкретної інформаційної системи. Шляхом внесення змін у конструктор класу FastAPI у головному файлі застосунку, було змінено назву проєкту на «ІС Замовлення їжі», додано технічний опис функціоналу та вказано актуальну версію API. Це забезпечує відповідність документації професійним стандартам і полегшує розуміння структури маршрутів для сторонніх розробників або тестувальників.

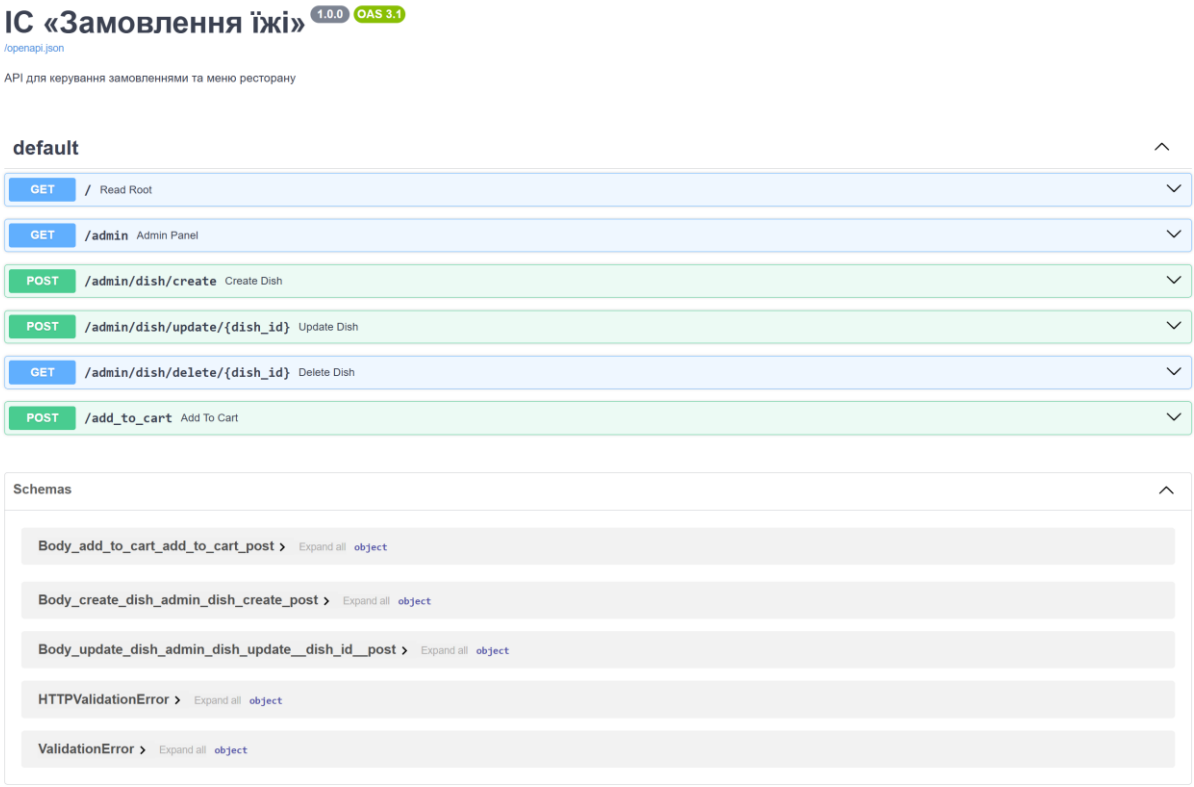

Рис 1.7 – Структура сторінки /docs

Висновок

У результаті виконання лабораторної роботи було розроблено вебзастосунок «ІС Замовлення їжі» на базі фреймворку FastAPI з використанням SQLAlchemy ORM для управління реляційною базою даних SQLite. Система підтримує серверний рендеринг сторінок через шаблонізатор Jinja2, розподіл прав між адміністратором та користувачем, а також повний цикл CRUD-операцій для керування меню. Розроблений проєкт повністю відповідає технічному завданню, демонструючи практичні навички побудови структурованих вебсистем та налаштування автоматичної документації OpenAPI.